Scheduling Edge Computing in the Presence of Wireless Interference

Maya Gutierrez
Autonomous Networks Research Group
University of Southern California
Los Angeles, CA, USA

Jared Coleman
Department of Computer Science
Loyola Marymount University
Los Angeles, CA, USA

Bhaskar Krishnamachari
Autonomous Networks Research Group
University of Southern California
Los Angeles, CA, USA

***Abstract*—Task-graph schedulers for heterogeneous edge computing, such as the Heterogeneous Earliest Finish Time (HEFT) algorithm, were developed under the assumption that link bandwidth is a static, decoupled property of each network edge. Wireless edge networks violate this assumption: simultaneously active links contend for spectrum, so each additional flow degrades the effective throughput of every other concurrently transmitting link. We show, through discrete-event simulation under an 802.11ax CSMA/CA interference model, that classical HEFT placements driven by widest-path bandwidth estimates spread tasks across the network and incur catastrophic multi-hop transfer costs that the scheduler's analytic estimate fails to anticipate; in some cases the actual makespan is two orders of magnitude worse than the prediction. A modified bandwidth matrix that imposes a heavy penalty on non-adjacent task pairs (HEFT-1) restores good performance by encouraging co-location and single-hop placement. Among nine routing schemes (three baselines, four static greedy, two dynamic), simple shortest-by-hop routing is the most robust winner across topologies and DAG sizes, with deferred dynamic routing (GSD-D) providing additional gains on sparse networks where bottleneck links must be time-shared rather than re-routed.**

***Index Terms*—edge computing, DAG scheduling, HEFT, wireless interference, CSMA/CA, routing, discrete-event simulation**

I. Introduction

Tactical and Internet-of-Battlefield-Things (IoBT) deployments increasingly push compute workloads onto wireless edge devices that are connected via multi-hop ad-hoc links rather than wired infrastructure [1], [2]. Many of these workloads admit a natural representation as a directed acyclic graph (DAG) of compute tasks with data dependencies, and the question of how to assign tasks to nodes and route the inter-task data flows is the classical problem of *DAG scheduling on heterogeneous platforms*. The Heterogeneous Earliest Finish Time (HEFT) algorithm of Topcuoglu et al. [3] remains the dominant list-scheduling heuristic for this setting and underpins many open implementations [4].

HEFT and its variants assume that the available bandwidth between any two nodes is a static scalar: a single number that depends only on the endpoints, not on what other transfers happen to be in flight at the same time. On wired clusters this is a serviceable approximation. On a shared wireless medium it is not: every active link consumes spectrum, and so the effective throughput of any one transfer collapses with the number of simultaneously active links in its neighborhood. Long multi-hop routes are particularly costly because each relay introduces an additional transmitter that interferes with all other concurrent flows.

This paper investigates how scheduler design must change when interference is modeled faithfully. We evaluate a discrete-event simulator (ncsim) of an 802.11ax wireless edge network using the closed-form Bianchi model [5] for CSMA/CA contention, and compare two HEFT variants and nine routing schemes across grids and random geometric graphs at multiple densities and DAG sizes. Our contributions are:

1) We show that an interference-oblivious bandwidth matrix (HEFT-2, widest-path between every pair) leads HEFT to spread tasks broadly, producing simulated makespans up to $727\times$ worse than its own analytic estimate (Section V-A).
2) We show that a bandwidth matrix that heavily penalizes non-adjacent task pairs (HEFT-1) drives HEFT toward co-location and adjacent placement, recovering most of the lost performance: between $1.7\times$ and $4\times$ better than HEFT-2 on the configurations we study.
3) Among nine routing schemes spanning three baselines, four static greedy variants, and two dynamic variants, the simplest interference-unaware shortest-by-hop (SH) rule wins the largest number of (network, DAG-size) configurations. Deferred dynamic routing (GSD-D) adds further gains on sparse networks where there are no good alternative paths (Section V-B).
4) We report auxiliary metrics (mean hop count, peak tasks per node, and peak link utilization) that explain *why* each combination wins (Section V-C).
5) We confirm via a no-interference baseline (Section V-D) that the headline effects collapse when interference is removed, isolating CSMA/CA contention as the mechanism behind the observations.

II. System Model

A. Network and DAG

A network is a set of compute nodes $\mathcal{N}$ and a set of wireless links $\mathcal{L} \subseteq \mathcal{N} \times \mathcal{N}$. Each node $n \in \mathcal{N}$ has a compute capacity C_n in compute-units per second; each link $\ell = (u, v)$ has a nominal bandwidth b_ℓ (MB/s) and propagation latency δ_ℓ (s). A workload is a DAG $G = (T, E)$: tasks $t \in T$ have compute

TABLE I
THE TWO HEFT BANDWIDTH-MATRIX VARIANTS COMPARED IN THIS PAPER.

Variant	Adjacent $(u, v) \in \mathcal{L}$	Non-adjacent
HEFT-2	widest-path PHY rate	widest-path PHY rate
HEFT-1	direct-link PHY rate	0.001 MB/s penalty

cost w_t (compute-units) and edges $(t, t') \in E$ carry data of size $d_{t,t'}$ (MB). When a task t runs on node n its execution time is w_t/C_n. When the data $d_{t,t'}$ traverses link ℓ alone it takes $d_{t,t'}/b_\ell + \delta_\ell$ seconds.

B. Bandwidth sharing and interference

Two effects modulate the nominal link bandwidth b_ℓ at runtime. First, *intra-link fair sharing*: if k flows traverse the same link, each receives a $1/k$ share of the link's bandwidth. Second, *inter-link interference*: simultaneously active links in the same neighborhood share the shared wireless medium. We use the closed-form Bianchi model [5] for IEEE 802.11ax DCF (5 GHz, 20 MHz channel, $P_{\text{tx}} = 20$ dBm) which yields, for each link ℓ and a set $\mathcal{A}$ of currently active links, an interference factor $f_\ell(\mathcal{A}) \in (0, 1]$. Composing the two effects, the effective per-flow throughput on link ℓ is

$$b_\ell^{\text{eff}} \;=\; \frac{b_\ell \, f_\ell(\mathcal{A})}{k_\ell}, \tag{1}$$

where k_ℓ is the number of flows on ℓ. As flows start and finish, the simulator updates $\mathcal{A}$ and recomputes b_ℓ^{eff} for all affected links.

C. HEFT and its bandwidth matrix

HEFT [3] computes a placement and an estimated finish time for each task by, at each step, placing the highest-upward-rank ready task on the node that minimizes its earliest finish time:

$$\text{EFT}(t, n) = \text{EST}(t, n) + \frac{w_t}{C_n}, \tag{2}$$

$$\text{EST}(t, n) = \max\Big(a_n, \max_{p \in \text{pred}(t)} \big[\text{EFT}(p, n_p) + d_{p,t}/R(n_p, n)\big]\Big), \tag{3}$$

where a_n is when node n next becomes available and $R(n_p, n)$ is the *pairwise bandwidth* HEFT assumes between nodes n_p and n. The function $R(\cdot, \cdot)$ is a static bandwidth matrix supplied to HEFT at the start of scheduling; the algorithm is unaware of any runtime contention.

The choice of $R(\cdot, \cdot)$ is a modeling decision, not part of the HEFT algorithm itself. We compare two reasonable choices, summarized in Table I.

HEFT-2 is the natural choice when one assumes multi-hop routes work roughly as efficiently as the widest available path. HEFT-1 encodes the prior that non-adjacent placement is essentially never desirable: the heavy penalty causes HEFT to co-locate or adjacent-place every communicating task pair.

TABLE II
ROUTING SCHEMES COMPARED. STATIC SCHEMES PLAN ALL ROUTES ONCE AT DAG INJECTION. DYNAMIC SCHEMES RECOMPUTE THE PATH AT EVERY TRANSFER-START EVENT.

Code	Class	Objective
W	baseline	widest-path: max $\min_\ell b_\ell$
S	baseline	shortest-path: min $\sum_\ell 1/b_\ell$
SH	baseline	shortest-hop, tie-break $\sum_\ell 1/b_\ell$
GS	stat. greedy	order flows by start time
GC	stat. greedy	order flows by upward-rank (ranku)
GB	stat. greedy	order flows by data size (largest first)
GO	stat. greedy	order flows by total temporal overlap
GSD	dynamic	score $\min_\ell b_\ell f_\ell/(k_\ell + 1)$
GSD-D	dyn. + defer	GSD; defer if eff./uncong. < 0.3

III. ROUTING SCHEMES

The scheduler decides *where* each task runs; a separate routing component decides *which links* carry each inter-task data flow. We compare nine schemes, listed in Table II. Three are baselines that ignore interference; four are static greedy schemes that pick paths to maximize total system throughput under HEFT's predicted concurrency; two are dynamic schemes that recompute paths at flow start time using the simulator's actual active-link state.

The four static greedy schemes share the same scoring function, evaluated when each candidate path is considered:

$$\text{score}(\text{path}) \;=\; \sum_{f \in \mathcal{F}_t} \min_{\ell \in \text{path}(f)} b_\ell^{\text{eff}}, \tag{4}$$

where $\mathcal{F}_t$ is the set of flows whose HEFT-predicted time windows overlap with the candidate's. The four variants differ only in the order in which flows are presented to the greedy planner.

The dynamic schemes (GSD, GSD-D) recompute the path inside the `transfer_start` event handler, using the actual set of currently active links and per-link transfer counts from the simulator. GSD picks the single path that maximizes its own bottleneck throughput (Eq. 1), ignoring siblings. GSD-D extends GSD with a deferral mechanism: if the newly chosen path's effective bottleneck bandwidth is below 30% of its uncongested value, the flow is postponed to the next `transfer_complete` event (with a starvation-prevention cap of three deferrals).

IV. EXPERIMENTAL SETUP

a) Simulator.: We use ncsim, an open-source headless discrete-event simulator that implements the model of Section II with microsecond event-time precision and the Bianchi-CSMA/CA interference model.[1]

b) Topologies.: We evaluate three classes of network. Two regular grids (4×4 and 7×7) with 40 m node spacing serve as controlled topologies. We also generate random geometric graphs of 50 nodes placed uniformly in an $L \times L$ square

[1] ncsim source and the scenarios used in this paper are available at https://github.com/ANRGUSC/iobt-ncsim.

TABLE III
HEFT-PREDICTED (ANALYTIC) VS. NCSIM-SIMULATED MAKESPAN IN SECONDS, FIXED SHORTEST-HOP (SH) ROUTING, MEAN OF 30 SEEDS. "S" = SMALL DAG (8 TASKS), "L" = LARGE DAG (30 TASKS). THE HEFT PREDICTION IS COMPUTED WITHOUT AN INTERFERENCE TERM.

Topology	HEFT-1		HEFT-2	
	predicted	ncsim	predicted	ncsim
4×4 S	11.0	18.3	10.6	93.9
4×4 L	4,021.9	292.4	23.9	1,071.7
7×7 S	10.9	20.3	10.2	93.8
7×7 L	18,039.0	614.2	24.8	2,492.1

with communication range $R = 80\,\text{m}$, varying L from 150 m (avg. degree 24.0, dense) through 500 m (avg. degree 3.7, sparse). Topology seeds are fixed per density level; only DAG / scheduler state randomization varies across replications.

c) DAG workloads.: For grid experiments we use a small DAG (8 tasks, fork-join) and a large DAG (30 tasks, 5-stage pipeline). For DAG-scaling experiments we vary task count from 8 to 60 (fork-join, 3-, 4-, 5-, and 6-stage pipelines). Compute costs are heterogeneous in $[150, 1000]$ cu, data sizes in $[2, 30]$ MB, node capacities in $[80, 300]$ cu/s.

d) Replication.: Mean makespan is reported with 95% confidence intervals over 20–30 random seeds per (topology, DAG, scheduler, routing) configuration.

V. RESULTS

A. Scheduler choice: HEFT-1 vs. HEFT-2 under interference

Table III compares the makespan HEFT *predicts* from its bandwidth matrix against the makespan ncsim actually *measures* when the same placement is executed under CSMA/CA interference and shortest-hop routing. Two facts stand out. First, HEFT-2's prediction (10–25 s) is systematically optimistic: simulation produces $4\times$–$100\times$ larger makespans because HEFT-2 assumes multi-hop routes are as efficient as the widest path, ignoring that each new active link reduces the effective throughput of every other simultaneously active link. Second, HEFT-1's prediction is wildly *pessimistic* on the large DAGs (4022 s for 4×4 L; 18039 s for 7×7 L) because the 0.001 MB/s non-adjacent penalty makes any multi-hop alternative look disastrous, forcing co-location and yielding what HEFT thinks is a severe compute-queuing bottleneck. In simulation, however, the compute bottleneck is a fraction of what HEFT estimates: tasks queue on a few high-capacity nodes but never compete for the wireless medium. The result is the rank inversion that drives this paper: HEFT-1 actually wins by 1.7–4$\times$ on the very experiments where HEFT-2's analytic estimate predicts a 168–727$\times$ HEFT-1 disadvantage.

The same effect is visible across the random-network density sweep (Fig. 2): HEFT-2's analytic estimate is essentially flat at all densities while the simulated makespan rises sharply as the graph sparsens and HEFT-2's preferred multi-hop routes lengthen.

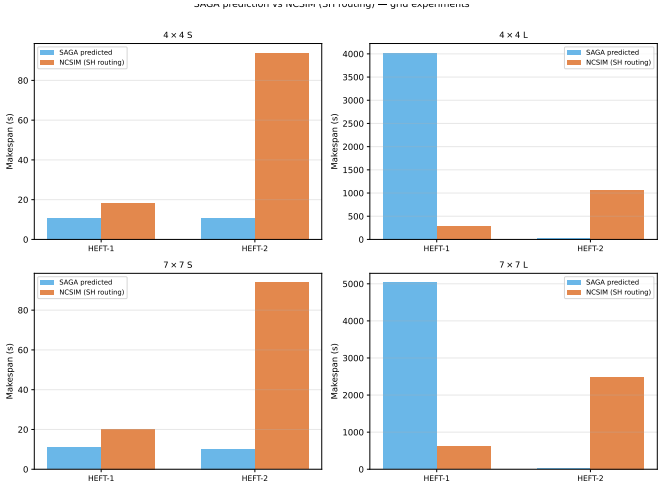

Fig. 1. HEFT analytic prediction (blue) vs. ncsim simulated makespan with SH routing (orange) for the four grid configurations of Table III. Each panel uses its own y-axis scale; the prediction–simulation gap inverts the HEFT-1/HEFT-2 ranking on both large-DAG cases.

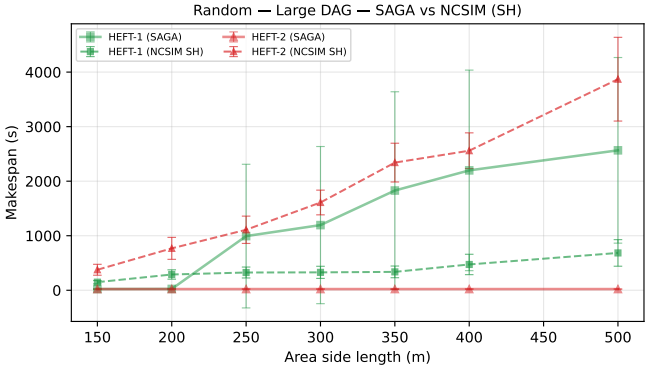

Fig. 2. Predicted (solid) vs. ncsim with SH routing (dashed) makespan as network density varies, large DAG, random geometric graphs. HEFT-2's prediction stays low at every density; the simulated values are an order of magnitude higher and grow as the network sparsens. HEFT-1's analytic estimate is more pessimistic but tracks the simulation more faithfully.

a) Mechanism.: HEFT-2 produces well load-balanced placements because widest-path bandwidth makes "put this task on a far node" look cheap; the resulting placement has many cross-node edges and thus many simultaneously active wireless links. Each of those links degrades every other link's effective bandwidth, and the global makespan is driven by the slowest of the resulting flows. HEFT-1's heavy non-adjacent penalty removes the "put it far away" option from the placement search, eliminating most cross-node data movement and the interference it would have generated.

B. Routing scheme comparison

Having fixed HEFT-1 as the placement strategy, we now ask which routing scheme to pair it with. We sweep DAG size from 8 to 60 tasks on three representative networks (L150 dense, L500 sparse, 7×7 grid) and report mean makespan over 20 seeds for each of the nine schemes. Per-network curves are

TABLE IV
NUMBER OF (NETWORK, DAG-SIZE) CELLS IN WHICH EACH ROUTING SCHEME ACHIEVES THE LOWEST MEAN MAKESPAN, OUT OF 18 CELLS TOTAL (3 NETWORKS × 6 DAG SIZES). HEFT-1 FIXED.

Routing	Total	L150	L500	7×7
SH	**6**	5	0	1
S	5	1	1	3
GSD-D	4	0	3	1
GB	2	0	1	1
GS	1	0	1	0
W, GC, GO, GSD	0	0	0	0

shown in Fig. 3; aggregate win counts across all 18 (network, DAG-size) cells appear in Table IV.

Three observations follow.

a) Dense networks: SH dominates and interference-aware greedy fails catastrophically.: On L150, SH wins at every DAG size from 16 tasks onward (S edges out SH at 8 tasks). The four static greedy schemes are an order of magnitude worse: 4,500–5,000 s at 60 tasks against SH's ∼412 s. Path diversity is the cause: on a graph with average degree 24 there are many alternative routes, and the greedy schemes choose paths that minimize *predicted* interference. That prediction is built from HEFT's already-stale concurrency assumptions; in execution the chosen paths trigger cascading recalculations that amplify actual interference. SH avoids this trap by simply minimizing the number of relay hops, which is by far the most important determinant of how many simultaneously active links the schedule produces.

b) Sparse networks: deferral wins.: On L500 (avg. degree 3.7), GSD-D wins three of six DAG sizes. With so few path alternatives, the choice of *path* is largely determined; the lever GSD-D pulls is *when* each flow runs. By postponing a flow whose effective throughput is severely degraded by current contention, GSD-D avoids double-booking the few bottleneck links and the additional makespan cost from the deferral itself is paid back several times over. Variance is high on L500 at the largest DAG sizes ($\sigma > \mu$ at 45–60 tasks), reflecting bimodal seed-to-seed behavior depending on whether HEFT placed any tasks across a bridge link.

c) Regular grid: shortest-path leads.: On the 7×7 grid, S leads at small to medium DAG sizes; SH and S are essentially tied on large DAGs. The grid has uniform link bandwidths, so the secondary $\sum 1/b_\ell$ tiebreaker that distinguishes S and SH rarely matters; what distinguishes both from the greedy schemes is again that they minimize the number of active links rather than try to predict interference. GSD (dynamic, no deferral) is the worst scheme on the grid, behind even the static greedy variants: per-flow path recomputation imposes overhead that degrades the very concurrency model it tries to optimize.

d) W is uniformly worst.: Widest-path routing wins zero of the 18 cells. Maximizing the bottleneck bandwidth pushes every flow toward the same handful of high-capacity links, inflating both the hop count of each route and the contention on those links. Both effects compound under CSMA/CA.

TABLE V
AUXILIARY METRICS FOR THE LARGE DAG (30 TASKS) ON REPRESENTATIVE RANDOM GEOMETRIC GRAPHS (50 NODES), AT THE BEST ROUTING SCHEME PER (DENSITY, SCHEDULER) CELL. "HOPS" IS THE MEAN HOP COUNT PER INTER-TASK TRANSFER; "PLU" IS THE PEAK LINK UTILIZATION; "MAKESPAN" IS IN SECONDS (MEAN OVER 30 SEEDS).

Network	Sched.	Routing	Hops	PLU	Makespan (s)
L150 (dense)	HEFT-1	SH	1.0	0.05	149.9
L150 (dense)	HEFT-2	SH	1.2	0.04	308.5
L300 (med.)	HEFT-1	SH	1.0	0.04	405.3
L300 (med.)	HEFT-2	SH	2.2	0.01	2,140.4
L500 (sparse)	HEFT-1	GSD-D	1.3	0.09	694.2
L500 (sparse)	HEFT-2	GSD-D	5.3	0.01	5,068.6

C. Auxiliary metrics: explaining why each combination wins

Table V and Figs. 4–5 make the mechanism quantitative. Across all densities, HEFT-1 drives the mean hop count to one; essentially every transfer either vanishes into a co-located edge (zero hops) or traverses a single direct wireless link. HEFT-2, which considers any pair of nodes as roughly bandwidth-equivalent, ends up routing flows over 1.2–5.3 hops on average. The peak link utilization tells the same story from a different angle: PLU is *lower* under HEFT-1 only because the schedule simply does not produce that many concurrent flows. Under HEFT-2 the few links that do see traffic see a lot of it, but the total number of active links is so much higher that the aggregate interference is severe even though no individual link is heavily loaded by its own flow alone. In other words, the relevant quantity is not "how busy is the busiest link" but "how many links are simultaneously busy."

D. No-interference baseline

To verify that the effects above are caused by inter-link interference rather than by raw bandwidth or hop count, we re-ran the random-network density sweep with the interference model disabled (i.e., with $f_\ell \equiv 1$ in Eq. 1 but with intra-link fair sharing preserved). Fig. 6 shows the result. With interference off, HEFT-1's makespan stays in the 47–62 s range across all densities and HEFT-2's is in the 70–145 s range, a $\sim 2\times$ gap rather than the $\sim 100\times$ gap of the interference case. The routing-scheme spread shrinks similarly. We conclude that inter-link interference is responsible for the headline effects of this paper, not any bandwidth-modeling or hop-count artifact.

VI. RELATED WORK

HEFT [3] and its many variants (CPOP, lookahead-HEFT, etc.) form the dominant family of heuristics for DAG scheduling on heterogeneous platforms, and the SAGA framework [4] provides reference implementations for benchmarking. However, these works adopt a static pairwise-bandwidth model that does not capture interference, and so the placements they produce are calibrated to a quantity that does not exist in a real wireless edge.

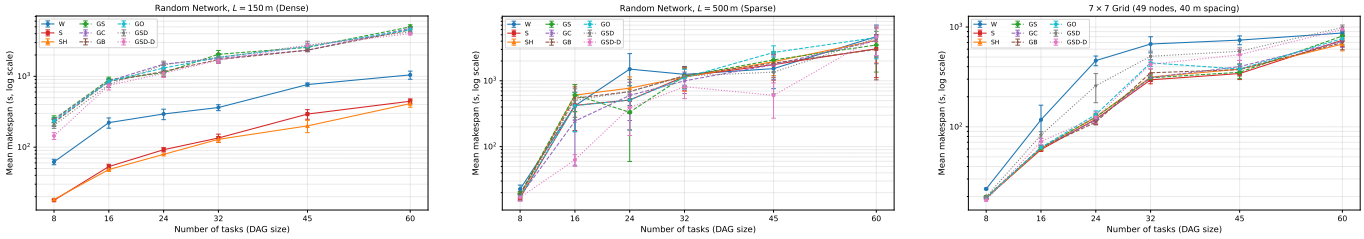

Fig. 3. Mean makespan (log scale) vs. DAG size for nine routing schemes, HEFT-1 scheduler, csma_bianchi interference. Left: dense random L150 (50 nodes, avg. degree 24.0). Middle: sparse random L500 (50 nodes, avg. degree 3.7). Right: regular 7×7 grid. Means over 20 seeds.

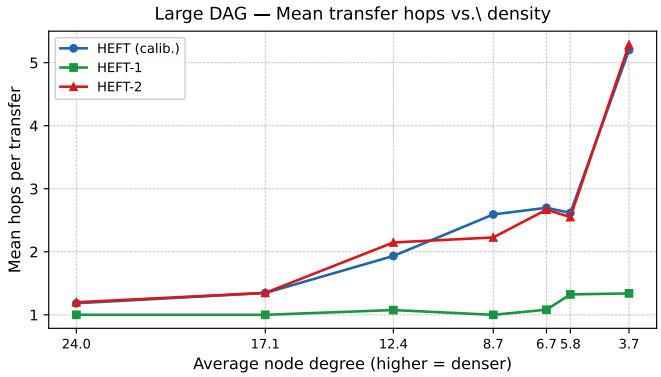

Fig. 4. Mean number of hops per inter-task transfer, large DAG, best routing scheme per (density, scheduler) cell. HEFT-1 stays at one hop across the entire density sweep; HEFT-2 and the calibrated HEFT baseline grow steeply as the graph sparsens and adjacent placement becomes less feasible.

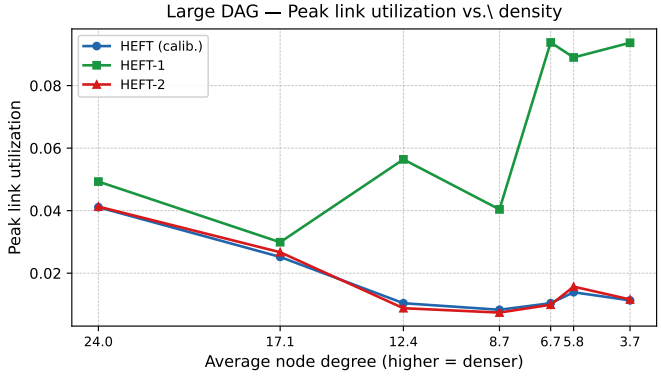

Fig. 5. Peak link utilization, large DAG, same configurations as Fig. 4. HEFT-1 keeps peak utilization low because most data transfers are eliminated by co-location, not because each link is underused.

The wireless networking literature characterizes 802.11 CSMA/CA contention in detail; the closed-form Bianchi model [5] remains the standard analytical baseline. Interference-aware routing in ad hoc networks is a long-standing topic. However, the schedulers it supplies typically optimize a single point-to-point flow rather than the joint placement-and-routing problem of a DAG workload, and the interaction between placement and routing under interference has, to our knowledge, not been systematically evaluated for the HEFT family on

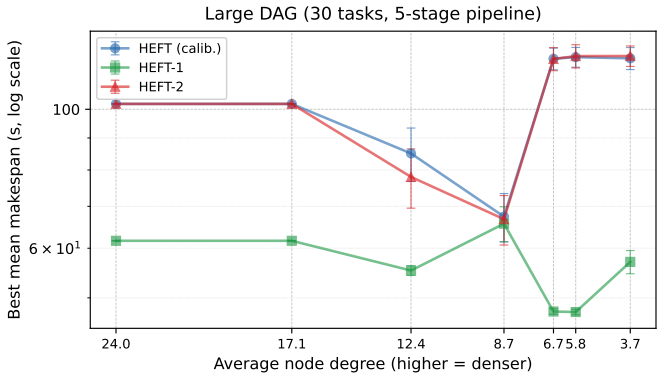

Fig. 6. Best-routing makespan vs. avg. degree, large DAG, with the inter-link interference model disabled (intra-link fair sharing only). The HEFT-1/HEFT-2 gap collapses from $\sim 100\times$ to roughly $2\times$, confirming that CSMA/CA contention is the mechanism behind the headline effects.

representative wireless edge topologies and DAG sizes.

VII. Conclusion

In this paper, we have shown that wireless interference inverts the intuition on which interference-oblivious HEFT scheduling rests: a placement that looks well load-balanced under a widest-path bandwidth model produces simultaneously active links whose mutual interference governs the makespan. A simple modification, a heavy bandwidth penalty on non-adjacent task pairs (HEFT-1), restores performance by encouraging single-hop placement. Among the nine routing schemes we evaluated, the simplest interference-unaware shortest-by-hop rule wins broadly; deferred dynamic routing (GSD-D) provides additional gains where sparse topologies leave no good alternative paths and the only useful lever is timing. From a system-design standpoint, these findings argue against investing in elaborate interference-aware routing on dense wireless edge deployments, where the bigger lever is to keep flows short and few.

In future work, we plan to extend the evaluation in three directions. First, we will model mobility, where the link set evolves during DAG execution and routing decisions must adapt online. Second, we will consider multi-DAG workloads in which placement and routing decisions must trade off across concurrent jobs. Third, we will compare the implicit co-location bias of HEFT-1 against placement heuristics that model interference explicitly inside the EFT computation.

REFERENCES

[1] A. Kott, A. Swami, and B. J. West, "The Internet of battle things," *Computer*, vol. 49, no. 12, pp. 70–75, 2016.

[2] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[3] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[4] J. Coleman, B. Krishnamachari *et al.*, "SAGA: Scheduling algorithm gathering and analysis," Open-source software, 2024, https://github.com/ANRGUSC/anrg-saga.

[5] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, Mar. 2000.